

Comparative Study of PINNs, XPINNs, and Quantum-Assisted PINNs for Burgers and Poisson Equations

Machine Learning For Physical Sciences Project Report

Supervisor: Dr. Sangkha Borah

Edara Yaswanth Balaji (EP22BTECH11007)

Vaishnavi Vyasraj (EP22BTECH11029)

Hamiz Mohammed C (EP22BTECH11009)

April 2026

This project solves and compares two partial differential equation benchmarks: the one-dimensional viscous Burgers equation and the two-dimensional Poisson equation. The active comparison is restricted to five neural models for each equation: a classical PINN, two XPINNs with different domain splits, a shallow QA-PINN, and a deeper QA-PINN.

Google Colab notebook: [Click Here](#)

Abstract

This report presents a focused comparison of classical and quantum-assisted physics-informed neural methods for solving two benchmark partial differential equations. The first problem is the one-dimensional viscous Burgers equation,

$$u_t + uu_x - \nu u_{xx} = 0, \quad x \in [-1, 1], \quad t \in [0, 1],$$

with initial condition $u(x, 0) = -\sin(\pi x)$, boundary conditions $u(-1, t) = u(1, t) = 0$, and viscosity $\nu = 0.01/\pi$. The second problem is the two-dimensional Poisson equation,

$$u_{xx} + u_{yy} = -2\pi^2 \sin(\pi x) \sin(\pi y), \quad (x, y) \in [0, 1]^2,$$

with homogeneous Dirichlet boundary conditions and exact solution $u(x, y) = \sin(\pi x) \sin(\pi y)$.

The five tested models are: (1) a classical physics-informed neural network (PINN), (2) an XPINN with the first domain split, (3) an XPINN with the second domain split, (4) a shallow quantum-assisted PINN (QA-PINN) with fewer quantum layers and a small classical head, and (5) a deeper QA-PINN with more quantum layers and a deeper classical head. Both QA-PINN variants are trained for more epochs than the classical PINN and XPINNs to give the quantum-assisted models a fairer opportunity despite the additional cost of quantum circuit simulation.

The numerical results show that model performance depends strongly on the equation. For Burgers, the space-split XPINN gives the lowest relative L^2 error among the five models, while the deep QA-PINN and classical PINN are close. For Poisson, the shallow QA-PINN achieves the best relative L^2 error, followed by the classical PINN. The report includes comparisons of loss, accuracy, training time, number of epochs, and model size. The central conclusion is that no single architecture is universally best; the most efficient model depends on the PDE structure, smoothness of the solution, domain decomposition, and computational budget.

Contents

1	Introduction and Problem Definition	1
1.1	Motivation	1
1.2	Exact Problem Statement	2
1.3	Five Models Compared	2
2	Mathematical Background	4
2.1	Partial Differential Equations	4
2.2	Burgers Equation	4
2.3	Poisson Equation	5
2.4	Why the Equations Are Complementary	5
3	Classical PINNs	7
3.1	PINN Principle	7
3.2	PINN Loss	7
3.3	Automatic Differentiation	8
4	Quantum-Assisted PINNs	10
4.1	Quantum Computing Basics	10
4.2	Variational Quantum Circuits	11
4.3	QA-PINN Literature	12
4.4	QA-PINN Results Preview	12
5	XPINNs	14
5.1	Domain Decomposition	14
5.2	Interface Loss	14
5.3	Two Splits Used	15
6	Experimental Setup	17
6.1	Training Budgets	17
6.2	Metrics	17

6.3	Why Model Size Matters	18
7	Burgers Results	19
7.1	Numerical Table	19
7.2	Burgers Prediction Plots	20
7.3	Burgers Comparison Plots	21
8	Poisson Results	23
8.1	Numerical Table	23
8.2	Poisson Prediction Plots	24
8.3	Poisson Comparison Plots	25
9	Discussion	27
9.1	Burgers Interpretation	27
9.2	Poisson Interpretation	27
9.3	Accuracy Versus Efficiency	27
9.4	Role of Model Size	28
10	Limitations and Future Work	29
10.1	Limitations	29
10.2	Future Work	29
11	Conclusion	31

Chapter 1

Introduction and Problem Definition

1.1 Motivation

Partial differential equations are fundamental tools for describing physical systems. Fluid motion, diffusion, electrostatics, elasticity, quantum mechanics, and heat transfer are all naturally expressed using PDEs. Classical numerical methods solve PDEs by discretizing the domain and approximating derivatives with finite differences, finite volumes, finite elements, or spectral operators. These methods are reliable, mathematically mature, and often highly optimized.

Scientific machine learning offers a different viewpoint. Instead of approximating the solution only at mesh nodes, a neural network can be used as a continuous function approximator. If the neural network is differentiable, automatic differentiation can compute the derivatives appearing in the PDE. The equation itself can then be placed into the loss function. This is the idea behind physics-informed neural networks, introduced in their modern form by Raissi, Perdikaris, and Karniadakis [1]. The broader field of physics-informed machine learning is reviewed by Karniadakis and coauthors [2].

In a general strong-form PDE, the unknown $u : \Omega \rightarrow \mathbb{R}$ satisfies

$$\mathcal{N}[u](\mathbf{x}) = 0, \quad \mathbf{x} \in \Omega, \quad \mathcal{B}[u](\mathbf{x}) = g(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega, \quad (1.1)$$

where $\mathcal{N}$ is the differential operator and $\mathcal{B}$ is the boundary or initial operator. A neural surrogate replaces u by u_θ and minimizes the sampled residual

$$\min_{\theta} \left[\frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{N}[u_\theta](\mathbf{x}_i^f)|^2 + \frac{\lambda}{N_b} \sum_{i=1}^{N_b} |\mathcal{B}[u_\theta](\mathbf{x}_i^b) - g(\mathbf{x}_i^b)|^2 \right]. \quad (1.2)$$

This report compares neural PDE solvers on two equations. The comparison is intentionally restricted to five methods: PINN, two XPINN splits, and two QA-PINN variants. This restriction is important because the goal is not to list many unrelated algorithms, but to compare related physics-informed models under a controlled experimental setup.

1.2 Exact Problem Statement

The first equation is the viscous Burgers equation:

$$u_t + uu_x - \nu u_{xx} = 0, \quad x \in [-1, 1], \quad t \in [0, 1], \quad (1.3)$$

$$u(x, 0) = -\sin(\pi x), \quad (1.4)$$

$$u(-1, t) = 0, \quad u(1, t) = 0, \quad (1.5)$$

$$\nu = \frac{0.01}{\pi}. \quad (1.6)$$

The Burgers equation contains nonlinear convection through uu_x and diffusion through νu_{xx} . This makes it a compact but meaningful model for nonlinear transport and viscous smoothing. Equivalently, since

$$\frac{\partial}{\partial x} \left(\frac{u^2}{2} \right) = uu_x, \quad (1.7)$$

the PDE can be written in conservative viscous form as

$$u_t + \left(\frac{u^2}{2} \right)_x = \nu u_{xx}. \quad (1.8)$$

The second equation is the Poisson equation:

$$u_{xx} + u_{yy} = -2\pi^2 \sin(\pi x) \sin(\pi y), \quad (x, y) \in [0, 1]^2, \quad (1.9)$$

$$u(x, y) = 0, \quad (x, y) \in \partial[0, 1]^2. \quad (1.10)$$

The exact solution is

$$u(x, y) = \sin(\pi x) \sin(\pi y). \quad (1.11)$$

This second problem is smooth, elliptic, and exactly verifiable. The forcing follows directly from

$$u_x = \pi \cos(\pi x) \sin(\pi y), \quad u_{xx} = -\pi^2 \sin(\pi x) \sin(\pi y), \quad (1.12)$$

$$u_y = \pi \sin(\pi x) \cos(\pi y), \quad u_{yy} = -\pi^2 \sin(\pi x) \sin(\pi y), \quad (1.13)$$

so that $\Delta u = u_{xx} + u_{yy} = -2\pi^2 \sin(\pi x) \sin(\pi y)$.

1.3 Five Models Compared

The active comparison uses the following models:

1. **PINN**: a classical fully connected neural network trained with PDE residual and boundary or initial-condition losses.
2. **XPINN split 1**: for Burgers this is a time split; for Poisson this is an x -split.
3. **XPINN split 2**: for Burgers this is a space split; for Poisson this is a y -split.
4. **QA-PINN shallow**: a quantum-assisted PINN with one quantum entangling layer and a small classical head, trained for 2000 epochs.

5. **QA-PINN deep**: a quantum-assisted PINN with three quantum entangling layers and a deeper classical head, also trained for 2000 epochs.

Tensor-network methods are not included in the final comparison. They were removed so the study focuses entirely on PINN-family and quantum-assisted PINN-family methods. All five models are evaluated as maps

$$u_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad (x, t) \mapsto u_\theta(x, t) \quad \text{or} \quad (x, y) \mapsto u_\theta(x, y), \quad (1.14)$$

and are compared with the same discrete error functional

$$E(\theta) = \left(\frac{\sum_i |u_\theta(\mathbf{x}_i) - u^*(\mathbf{x}_i)|^2}{\sum_i |u^*(\mathbf{x}_i)|^2} \right)^{1/2}. \quad (1.15)$$

Physics-informed neural network workflow

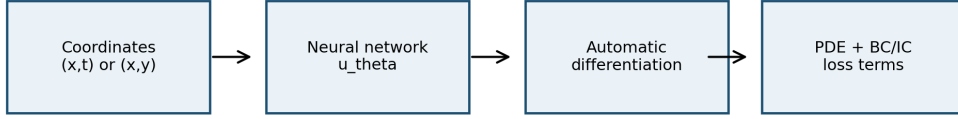


Figure 1.1: Physics-informed neural network workflow used as the conceptual foundation for all five models.

Chapter 2

Mathematical Background

2.1 Partial Differential Equations

A partial differential equation relates an unknown multivariable function to its partial derivatives. PDEs can be classified by linearity, order, and type. A PDE is linear if the unknown function and its derivatives appear linearly. It is nonlinear if products or nonlinear functions of the unknown appear. The Burgers equation is nonlinear because of the product uu_x . The Poisson equation is linear because u appears only through its second derivatives.

PDEs are also classified as hyperbolic, parabolic, or elliptic. Hyperbolic equations describe wave-like propagation. Parabolic equations describe diffusion-like evolution. Elliptic equations describe steady global balance. Burgers with viscosity is a nonlinear convection-diffusion equation, while Poisson is elliptic. For a second-order linear PDE in two variables,

$$Au_{xx} + 2Bu_{xy} + Cu_{yy} + Du_x + Eu_y + Fu = G, \quad (2.1)$$

the discriminant

$$\Delta_{\text{PDE}} = B^2 - AC \quad (2.2)$$

classifies the equation locally: $\Delta_{\text{PDE}} > 0$ is hyperbolic, $\Delta_{\text{PDE}} = 0$ is parabolic, and $\Delta_{\text{PDE}} < 0$ is elliptic. For Poisson, $A = C = 1$ and $B = 0$, hence $\Delta_{\text{PDE}} = -1 < 0$.

2.2 Burgers Equation

The Burgers equation is a standard benchmark because it is simple to write but difficult enough to test numerical methods. The nonlinear term uu_x transports the solution at a speed depending on the solution itself. The diffusion term νu_{xx} smooths the solution. When viscosity is small, steep gradients can develop before being smoothed.

The inviscid form can be written as

$$u_t + \left(\frac{u^2}{2} \right)_x = 0, \quad (2.3)$$

which is a scalar conservation law. This form shows that Burgers equation is related to conservation of a flux. The viscous term makes the equation more stable and prevents

true discontinuous shocks. Multiplying the viscous equation by u and integrating over the spatial domain gives the energy balance

$$\frac{1}{2} \frac{d}{dt} \int_{-1}^1 u^2 dx = - \int_{-1}^1 u^2 u_x dx + \nu \int_{-1}^1 u u_{xx} dx \quad (2.4)$$

$$= - \left[\frac{u^3}{3} \right]_{-1}^1 - \nu \int_{-1}^1 u_x^2 dx \quad (2.5)$$

$$= -\nu \int_{-1}^1 u_x^2 dx \leq 0, \quad (2.6)$$

using $u(-1, t) = u(1, t) = 0$. This confirms that viscosity dissipates energy.

2.3 Poisson Equation

The Poisson equation

$$\Delta u = f \quad (2.7)$$

appears in electrostatics, gravitation, pressure projection, and steady heat conduction. In two dimensions,

$$\Delta u = u_{xx} + u_{yy}. \quad (2.8)$$

The manufactured solution in this project is smooth and separable:

$$u(x, y) = \sin(\pi x) \sin(\pi y). \quad (2.9)$$

The forcing is obtained by differentiating the exact solution:

$$u_{xx} + u_{yy} = -2\pi^2 \sin(\pi x) \sin(\pi y). \quad (2.10)$$

The boundary condition is satisfied because

$$u(0, y) = u(1, y) = u(x, 0) = u(x, 1) = 0, \quad (2.11)$$

since $\sin(0) = \sin(\pi) = 0$. Thus the residual for the exact solution is

$$r^*(x, y) = u_{xx}^* + u_{yy}^* + 2\pi^2 \sin(\pi x) \sin(\pi y) = 0. \quad (2.12)$$

2.4 Why the Equations Are Complementary

Burgers and Poisson test different capabilities. Burgers tests nonlinear time-dependent learning and sensitivity to steep gradients. Poisson tests smooth elliptic residual learning in two spatial dimensions. A model that works well for one is not guaranteed to work best for the other. This makes the pair useful for a comparative machine-learning study. In operator form, the two residuals are

$$\mathcal{N}_B[u] = u_t + uu_x - \nu u_{xx}, \quad (2.13)$$

$$\mathcal{N}_P[u] = u_{xx} + u_{yy} + 2\pi^2 \sin(\pi x) \sin(\pi y). \quad (2.14)$$

The Burgers residual contains a nonlinear product uu_x , while the Poisson residual is linear in u . This difference changes the optimization landscape seen by the same neural architecture.

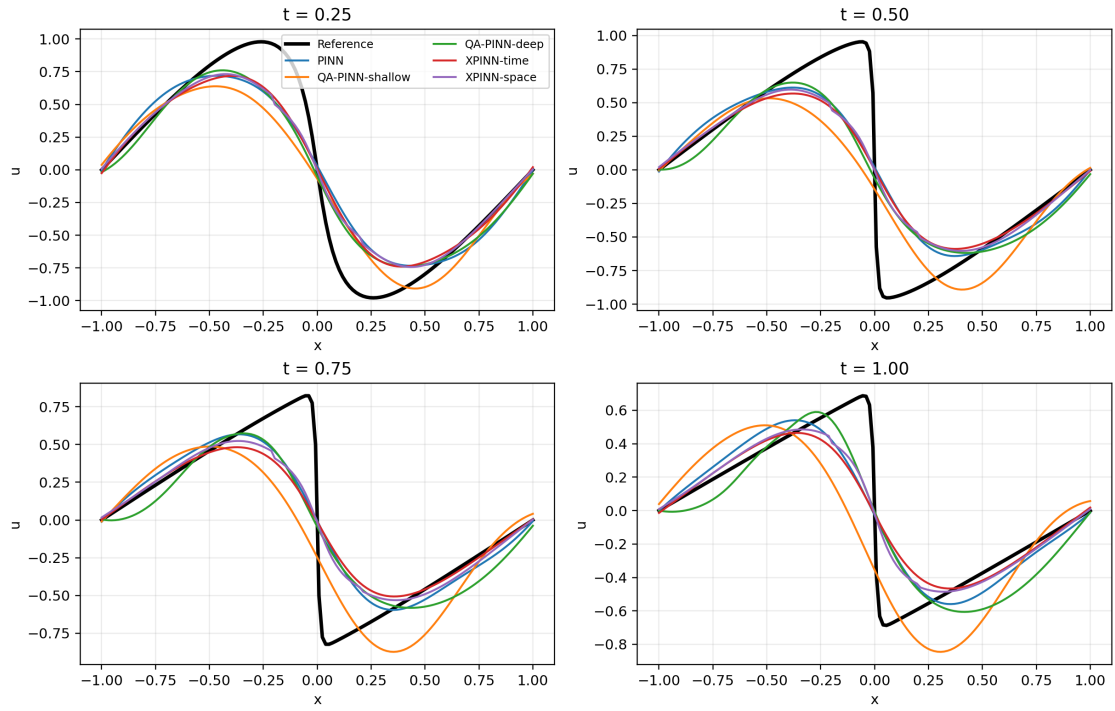


Figure 2.1: Burgers reference and model predictions on selected time slices.

Chapter 3

Classical PINNs

3.1 PINN Principle

A physics-informed neural network approximates the PDE solution using a neural network u_θ , where θ denotes trainable weights and biases. Instead of training only on labeled data, the network is trained to satisfy the governing equation. This is done by evaluating the PDE residual at collocation points.

For Burgers, the residual is

$$f_\theta(x, t) = u_{\theta,t} + u_\theta u_{\theta,x} - \nu u_{\theta,xx}. \quad (3.1)$$

For Poisson, the residual is

$$r_\theta(x, y) = u_{\theta,xx} + u_{\theta,yy} + 2\pi^2 \sin(\pi x) \sin(\pi y). \quad (3.2)$$

If the network exactly satisfies the PDE, these residuals are zero. For a multilayer perceptron with L hidden layers,

$$\mathbf{h}^{(0)} = \mathbf{x}, \quad (3.3)$$

$$\mathbf{h}^{(\ell)} = \tanh(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \ell = 1, \dots, L, \quad (3.4)$$

$$u_\theta(\mathbf{x}) = W^{(L+1)}\mathbf{h}^{(L)} + b^{(L+1)}. \quad (3.5)$$

The PDE residual is therefore a differentiable function of both the coordinates and the trainable parameters.

3.2 PINN Loss

For Burgers, the loss combines PDE, initial-condition, and boundary losses:

$$\mathcal{L} = \mathcal{L}_{PDE} + \lambda(\mathcal{L}_{IC} + \mathcal{L}_{BC}). \quad (3.6)$$

For Poisson, the loss combines PDE and boundary losses:

$$\mathcal{L} = \mathcal{L}_{PDE} + \lambda_b \mathcal{L}_{BC}. \quad (3.7)$$

The weights λ and λ_b control the relative importance of physics residuals and boundary data. The Burgers terms are evaluated as

$$\mathcal{L}_{PDE} = \frac{1}{N_f} \sum_{i=1}^{N_f} |u_{\theta,t}(x_i, t_i) + u_{\theta}(x_i, t_i)u_{\theta,x}(x_i, t_i) - \nu u_{\theta,xx}(x_i, t_i)|^2, \quad (3.8)$$

$$\mathcal{L}_{IC} = \frac{1}{N_i} \sum_{i=1}^{N_i} |u_{\theta}(x_i, 0) + \sin(\pi x_i)|^2, \quad (3.9)$$

$$\mathcal{L}_{BC} = \frac{1}{N_b} \sum_{i=1}^{N_b} (|u_{\theta}(-1, t_i)|^2 + |u_{\theta}(1, t_i)|^2). \quad (3.10)$$

For Poisson,

$$\mathcal{L}_{PDE} = \frac{1}{N_f} \sum_{i=1}^{N_f} |u_{\theta,xx}(x_i, y_i) + u_{\theta,yy}(x_i, y_i) + 2\pi^2 \sin(\pi x_i) \sin(\pi y_i)|^2, \quad (3.11)$$

$$\mathcal{L}_{BC} = \frac{1}{N_b} \sum_{i=1}^{N_b} |u_{\theta}(x_i^b, y_i^b)|^2, \quad (x_i^b, y_i^b) \in \partial[0, 1]^2. \quad (3.12)$$

3.3 Automatic Differentiation

Automatic differentiation is essential for PINNs. The network output is differentiated with respect to its input coordinates to compute u_t , u_x , u_{xx} , and u_{yy} . The present implementation uses smooth tanh activations because second derivatives are required. A nonsmooth activation such as ReLU is less appropriate for strong-form second-order PDE residuals because its second derivative is zero almost everywhere and undefined at the kink. For a scalar output $u_{\theta}(\mathbf{x})$, automatic differentiation gives

$$\nabla_{\mathbf{x}} u_{\theta} = \left[\frac{\partial u_{\theta}}{\partial x_1}, \dots, \frac{\partial u_{\theta}}{\partial x_d} \right], \quad H_{\theta} = \nabla_{\mathbf{x}}^2 u_{\theta}. \quad (3.13)$$

The second derivatives used by the residual are diagonal entries of the Hessian:

$$u_{xx} = e_x^{\top} H_{\theta} e_x, \quad u_{yy} = e_y^{\top} H_{\theta} e_y. \quad (3.14)$$

The Adam optimizer updates parameters approximately as

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}_k, \quad (3.15)$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}_k)^2, \quad (3.16)$$

$$\theta_{k+1} = \theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}. \quad (3.17)$$

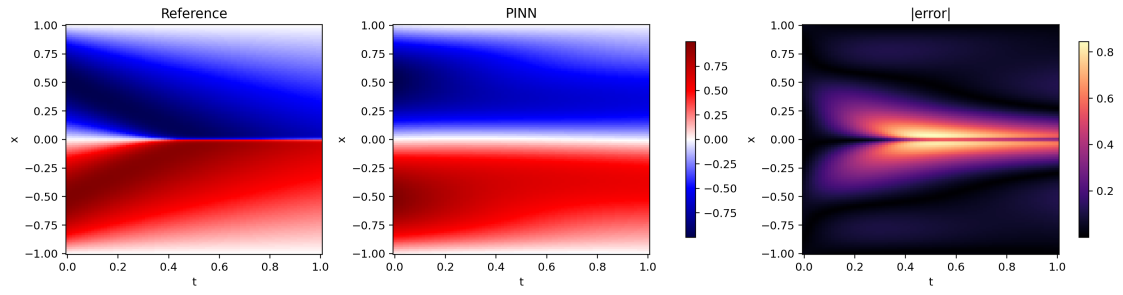


Figure 3.1: Classical PINN prediction and absolute error for the Burgers equation.

Chapter 4

Quantum-Assisted PINNs

4.1 Quantum Computing Basics

Quantum computing is based on quantum states, unitary evolution, and measurement. A classical bit is either 0 or 1. A qubit is a normalized superposition

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (4.1)$$

The complex coefficients α and β are probability amplitudes. A measurement in the computational basis returns 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$. The same state can be parameterized on the Bloch sphere as

$$|\psi\rangle = \cos\left(\frac{\vartheta}{2}\right)|0\rangle + e^{i\varphi}\sin\left(\frac{\vartheta}{2}\right)|1\rangle. \quad (4.2)$$

Single-qubit rotations used in angle embedding are unitary matrices such as

$$R_y(\alpha) = \begin{bmatrix} \cos(\alpha/2) & -\sin(\alpha/2) \\ \sin(\alpha/2) & \cos(\alpha/2) \end{bmatrix}, \quad R_z(\alpha) = \begin{bmatrix} e^{-i\alpha/2} & 0 \\ 0 & e^{i\alpha/2} \end{bmatrix}. \quad (4.3)$$

Multi-qubit systems can exhibit entanglement. An entangled state cannot be written as a simple product of individual qubit states. Entanglement is one of the features that gives quantum computing its expressive power. Quantum gates are unitary transformations that rotate and entangle qubits. In variational quantum circuits, some gate angles are trainable parameters.

Bloch sphere view of one qubit

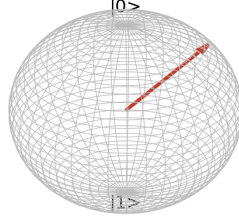


Figure 4.1: Bloch sphere representation of a single qubit state. A general qubit is a point on the sphere before measurement.

4.2 Variational Quantum Circuits

A variational quantum circuit has three stages: data encoding, trainable quantum evolution, and measurement. In this project, the input coordinates are first mapped to rotation angles. Angle embedding places the classical input into the quantum circuit. Basic entangling layers then apply trainable rotations and entangling gates. Finally, Pauli-Z expectation values are measured and passed to a classical neural head.

The hybrid structure is

coordinates $\rightarrow$ classical encoder $\rightarrow$ quantum circuit $\rightarrow$ expectation values $\rightarrow$ classical output head. (4.4)

This is called quantum-assisted rather than fully quantum because the optimizer, loss construction, sampling, and final output processing are classical. Mathematically, the encoder produces angles

$$\boldsymbol{\alpha} = \pi \tanh(W_e \mathbf{x} + \mathbf{b}_e), \quad (4.5)$$

and the variational circuit applies

$$|\psi(\mathbf{x}; \phi)\rangle = U_\phi U_{\text{enc}}(\boldsymbol{\alpha})|0\rangle^{\otimes n_q}. \quad (4.6)$$

The quantum feature vector is made of Pauli-Z expectations,

$$q_j(\mathbf{x}; \phi) = \langle \psi(\mathbf{x}; \phi) | Z_j | \psi(\mathbf{x}; \phi) \rangle, \quad j = 1, \dots, n_q, \quad (4.7)$$

and the final prediction is $u_\theta(\mathbf{x}) = h_\omega(\mathbf{q})$ with $\theta = (W_e, \mathbf{b}_e, \phi, \omega)$.

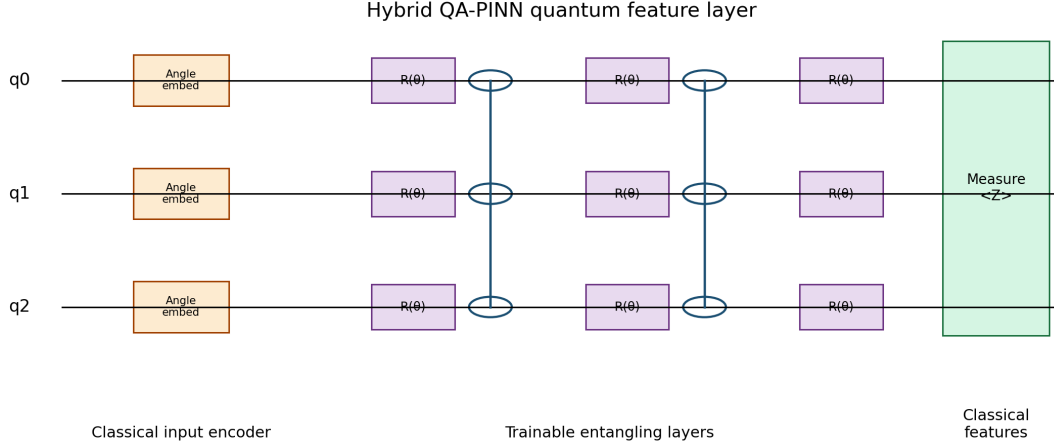


Figure 4.2: Schematic of the hybrid QA-PINN quantum feature layer.

4.3 QA-PINN Literature

Quantum PINNs have recently been studied as a way to combine variational quantum circuits with physics-informed losses. Trahan, Loveland, and Dent [5] studied quantum and hybrid quantum/classical PINNs for steady and transient PDEs using PennyLane. Farea, Khan, and Celebi [6] studied quantum-classical PINNs for solving PDEs and emphasized parameter-efficient performance. Recent hybrid quantum PINN work on flow problems [7] shows that quantum-assisted models can be promising but also problem dependent.

This project uses two QA-PINN variants. The shallow variant has fewer quantum layers and a smaller classical head. The deep variant has more quantum layers and a deeper classical head. Both are trained longer than the classical PINN and XPINNs to improve their chance of reaching efficient performance. Both variants minimize the same physics-informed objective as the classical PINN:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(u_{\theta}), \quad (4.8)$$

but the hypothesis class is changed by replacing intermediate classical features with quantum expectation values. The trainable parameter count is

$$P_{QA} = P_{\text{encoder}} + P_{\text{quantum}} + P_{\text{head}}, \quad (4.9)$$

which explains why the shallow QA-PINN can remain very small.

4.4 QA-PINN Results Preview

The shallow QA-PINN performs very well on Poisson, achieving the lowest relative L^2 error among all five models. This is plausible because the Poisson exact solution is sinusoidal, and quantum angle encodings naturally generate trigonometric features. On Burgers, the deep QA-PINN is much more accurate than the shallow QA-PINN and is close to the classical PINN. However, it requires significantly more training time because quantum circuit simulation is expensive on CPU. The sinusoidal compatibility can be seen from simple rotation

expectations:

$$\langle 0 | R_y(\alpha)^\dagger Z R_y(\alpha) | 0 \rangle = \cos \alpha, \quad (4.10)$$

so angle-based quantum features naturally produce trigonometric basis functions. This aligns well with $u^*(x, y) = \sin(\pi x) \sin(\pi y)$ after classical mixing in the output head.

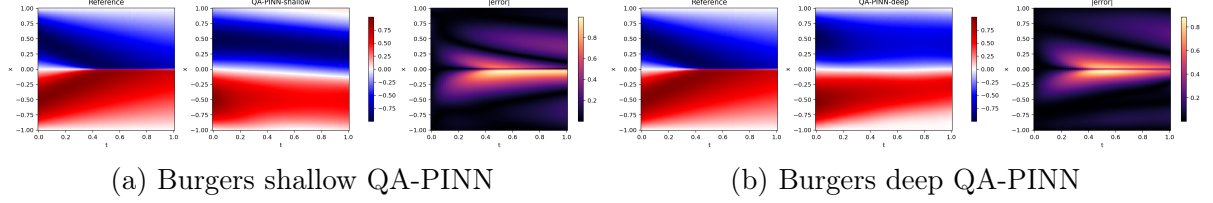


Figure 4.3: Two QA-PINN variants on the Burgers equation.

Chapter 5

XPINNs

5.1 Domain Decomposition

Extended physics-informed neural networks use domain decomposition. Instead of training one global network on the whole domain, the domain is split into subdomains and a separate neural network is trained in each subdomain. Jagtap and Karniadakis introduced XPINNs as a generalized space-time domain-decomposition framework for nonlinear PDEs [4]. XPINNs extend conservative PINN ideas [3] and can be applied beyond conservation laws.

Domain decomposition helps when different regions of the domain have different complexity. A subdomain with steep gradients can use more capacity or more points. A smooth region can use a smaller network. The main challenge is interface consistency: neighboring subnetworks must agree where they meet. Let the domain be decomposed as

$$\Omega = \bigcup_{k=1}^K \Omega_k, \quad \Omega_i \cap \Omega_j = \Gamma_{ij} \text{ for neighboring subdomains.} \quad (5.1)$$

An XPINN uses local subnetworks u_{θ_k} and the global prediction is piecewise:

$$u_{\Theta}(\mathbf{x}) = u_{\theta_k}(\mathbf{x}), \quad \mathbf{x} \in \Omega_k, \quad \Theta = \{\theta_1, \dots, \theta_K\}. \quad (5.2)$$

5.2 Interface Loss

The XPINN loss has three main parts:

$$\mathcal{L}_{XPINN} = \mathcal{L}_{PDE} + \lambda_D \mathcal{L}_{data} + \lambda_I \mathcal{L}_{interface}. \quad (5.3)$$

The interface term penalizes jumps in the solution value, jumps in the normal derivative, and disagreement in the PDE residual. For Burgers space splitting, the normal derivative is u_x . For Poisson x splitting, it is also u_x ; for Poisson y splitting, it is u_y . For an interface

Γ_{ij} with unit normal $\mathbf{n}$, the implemented penalty has the form

$$\mathcal{L}_{interface} = \frac{1}{N_\Gamma} \sum_{\mathbf{x}_m \in \Gamma_{ij}} |u_{\theta_i}(\mathbf{x}_m) - u_{\theta_j}(\mathbf{x}_m)|^2 \quad (5.4)$$

$$+ \frac{\alpha}{N_\Gamma} \sum_{\mathbf{x}_m \in \Gamma_{ij}} |\nabla u_{\theta_i}(\mathbf{x}_m) \cdot \mathbf{n} - \nabla u_{\theta_j}(\mathbf{x}_m) \cdot \mathbf{n}|^2 \quad (5.5)$$

$$+ \frac{\beta}{N_\Gamma} \sum_{\mathbf{x}_m \in \Gamma_{ij}} |\mathcal{N}[u_{\theta_i}](\mathbf{x}_m) - \mathcal{N}[u_{\theta_j}](\mathbf{x}_m)|^2. \quad (5.6)$$

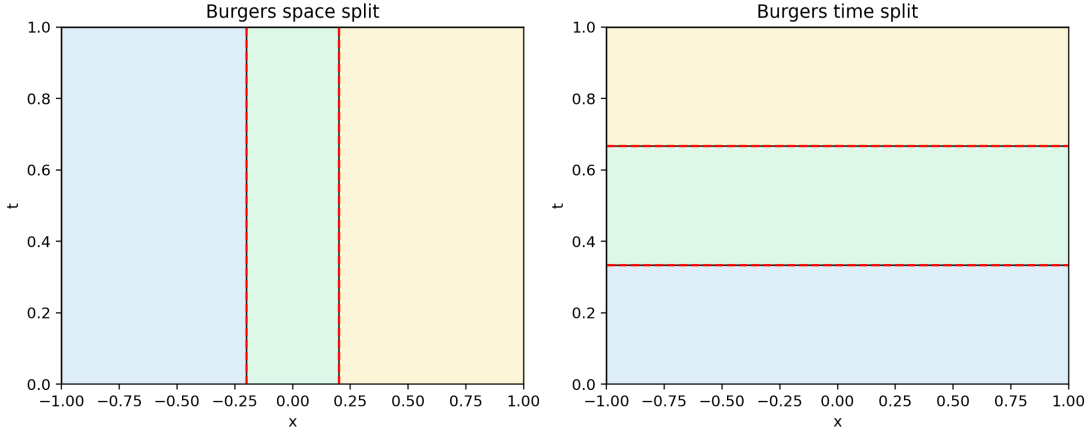


Figure 5.1: Schematic of Burgers XPINN space and time splits.

5.3 Two Splits Used

For Burgers, the two XPINN splits are:

- **Time split:** three time slabs over $t \in [0, 1]$.
- **Space split:** three spatial regions over $x \in [-1, 1]$, with a central region around the steepest solution feature.

For Poisson, the two XPINN splits are:

- **x split:** three vertical slabs in the square.
- **y split:** three horizontal slabs in the square.

For example, the Burgers time split uses

$$\Omega_1 = [-1, 1] \times [0, 1/3], \quad \Omega_2 = [-1, 1] \times [1/3, 2/3], \quad \Omega_3 = [-1, 1] \times [2/3, 1], \quad (5.7)$$

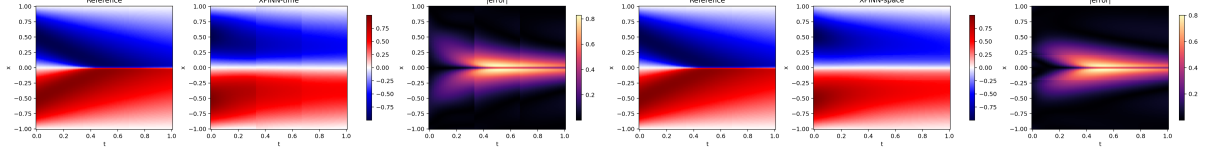
whereas the space split uses

$$\Omega_1 = [-1, -0.2] \times [0, 1], \quad \Omega_2 = [-0.2, 0.2] \times [0, 1], \quad \Omega_3 = [0.2, 1] \times [0, 1]. \quad (5.8)$$

For Poisson, the vertical split is

$$\Omega_1 = [0, 1/3] \times [0, 1], \quad \Omega_2 = [1/3, 2/3] \times [0, 1], \quad \Omega_3 = [2/3, 1] \times [0, 1], \quad (5.9)$$

with an analogous horizontal split in the y direction.



(a) Burgers XPINN time split

(b) Burgers XPINN space split

Figure 5.2: Burgers XPINN predictions and errors.

Chapter 6

Experimental Setup

6.1 Training Budgets

The final experiments use higher training budgets than the earlier draft. The classical PINN and XPINNs are trained for 1500 epochs. Both QA-PINN variants are trained for 2000 epochs. The longer QA-PINN schedule is used because the quantum-assisted models are more expensive per epoch and can require more optimization steps to stabilize.

For Burgers, the evaluation grid is 128×81 over (x, t) . For Poisson, the evaluation grid is 64×64 over (x, y) . All models are trained on CPU in the available environment. The same random seed is used for reproducibility within each experiment. The Burgers grid is

$$x_i = -1 + \frac{2(i-1)}{127}, \quad i = 1, \dots, 128, \quad t_j = \frac{j-1}{80}, \quad j = 1, \dots, 81. \quad (6.1)$$

The Poisson grid is

$$x_i = y_i = \frac{i-1}{63}, \quad i = 1, \dots, 64. \quad (6.2)$$

The Burgers finite-difference reference uses an explicit update of the form

$$u_i^{n+1} = u_i^n - \Delta t u_i^n D_x^{up} u_i^n + \nu \Delta t D_{xx} u_i^n, \quad (6.3)$$

where D_x^{up} is an upwind first derivative and D_{xx} is a centered second derivative.

6.2 Metrics

The reported metrics are:

$$\text{MSE} = \frac{1}{N} \sum_i (\hat{u}_i - u_i)^2, \quad (6.4)$$

$$\text{MAE} = \frac{1}{N} \sum_i |\hat{u}_i - u_i|, \quad (6.5)$$

$$L^\infty = \max_i |\hat{u}_i - u_i|, \quad (6.6)$$

$$\text{Relative } L^2 = \frac{\|\hat{u} - u\|_2}{\|u\|_2}. \quad (6.7)$$

The report also compares training time, number of epochs, and trainable parameter count. These cost metrics are essential because accuracy alone does not determine model efficiency. The discrete vector notation is

$$\mathbf{e} = \hat{\mathbf{u}} - \mathbf{u}, \quad \|\mathbf{e}\|_2 = \left(\sum_i e_i^2 \right)^{1/2}, \quad \|\mathbf{e}\|_\infty = \max_i |e_i|. \quad (6.8)$$

Thus the reported relative error is

$$\text{RelL2} = \frac{\|\mathbf{e}\|_2}{\|\mathbf{u}\|_2}. \quad (6.9)$$

6.3 Why Model Size Matters

The shallow QA-PINN has only 133 trainable parameters, while the deep QA-PINN has 7315. The classical PINN has 4897 parameters, and each XPINN has 10323 trainable parameters because it contains three subnetworks. A small model that achieves good accuracy is efficient in parameter count, but it may still be slow if each epoch is expensive. This is why both parameter count and wall-clock training time are reported. For a dense layer with input width m and output width n , the trainable parameter count is

$$P_{\text{dense}} = mn + n. \quad (6.10)$$

For an XPINN with $K = 3$ subdomains and identical subnetworks,

$$P_{\text{XPINN}} = \sum_{k=1}^3 P_{\theta_k} = 3P_{\text{subnet}}. \quad (6.11)$$

Runtime is not proportional only to P because QA-PINNs also pay the quantum-circuit simulation cost:

$$T_{\text{train}} \approx N_{\text{epochs}} (T_{\text{AD}} + T_{\text{circuit}} + T_{\text{optimizer}}). \quad (6.12)$$

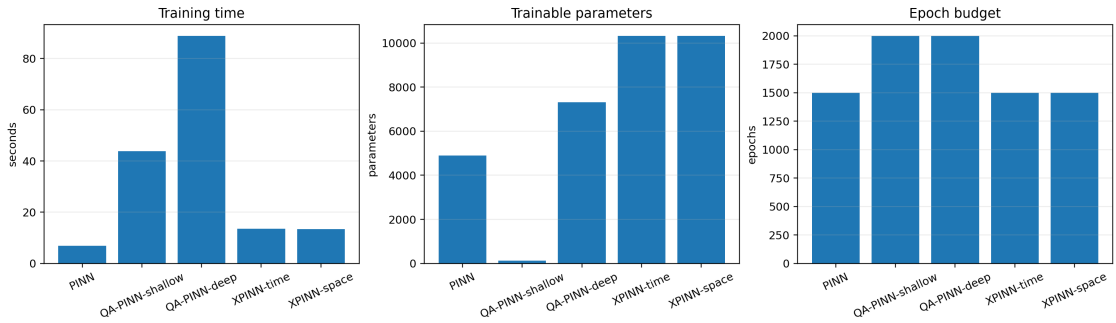


Figure 6.1: Burgers comparison of training time, trainable parameters, and epoch budgets.

Chapter 7

Burgers Results

7.1 Numerical Table

Table 7.1: Burgers errors and cost metrics for the five active models.

Model	Rel. L^2	MSE	Time (s)	Epochs	Params
PINN	0.3978	5.73×10^{-2}	6.9	1500	4897
QA-PINN shallow	0.4706	8.02×10^{-2}	43.8	2000	133
QA-PINN deep	0.3976	5.72×10^{-2}	88.8	2000	7315
XPINN time	0.4046	5.92×10^{-2}	13.6	1500	10323
XPINN space	0.3798	5.22×10^{-2}	13.4	1500	10323

The best Burgers model is the space-split XPINN. This agrees with the physical structure of the Burgers solution: the main difficulty is spatially localized near the central steepening region. The deep QA-PINN and classical PINN are close to each other in relative L^2 error. The shallow QA-PINN has far fewer parameters but is less accurate on Burgers. The ranking is determined by

$$\theta_{\text{best}} = \arg \min_{\theta \in \Theta_{\text{models}}} \frac{\|\hat{u}_{\theta} - u_{\text{FD}}\|_2}{\|u_{\text{FD}}\|_2}, \quad (7.1)$$

which gives the space-split XPINN for the Burgers benchmark. Its residual target is

$$f_{\theta_k}(x, t) = u_{\theta_k, t} + u_{\theta_k} u_{\theta_k, x} - \nu u_{\theta_k, xx} = 0, \quad (x, t) \in \Omega_k, \quad (7.2)$$

with interface continuity at $x = -0.2$ and $x = 0.2$.

7.2 Burgers Prediction Plots

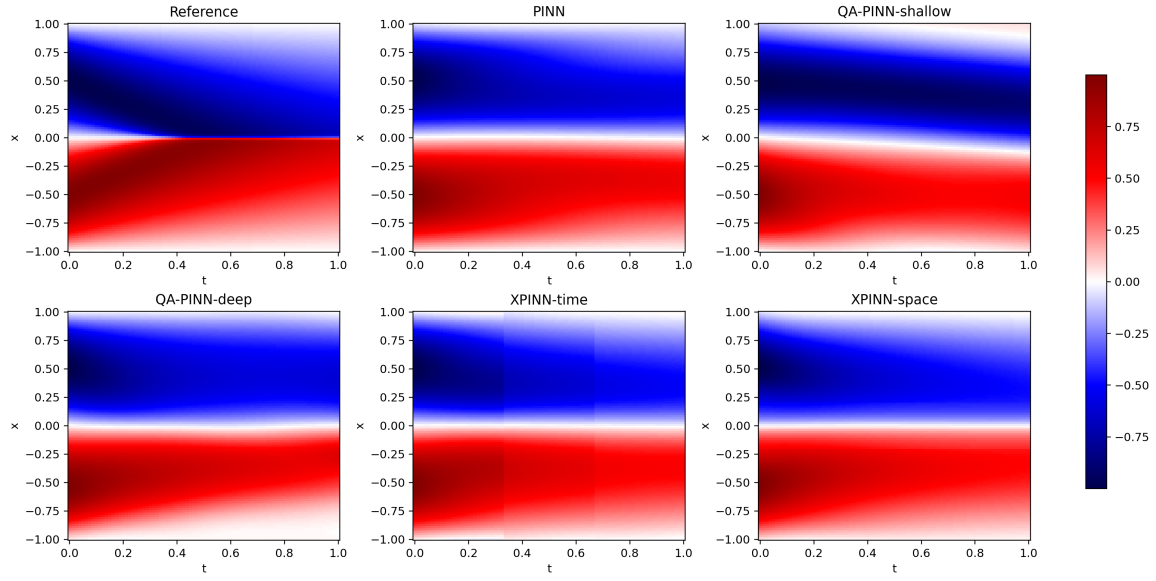


Figure 7.1: Burgers prediction montage for reference and all five models.

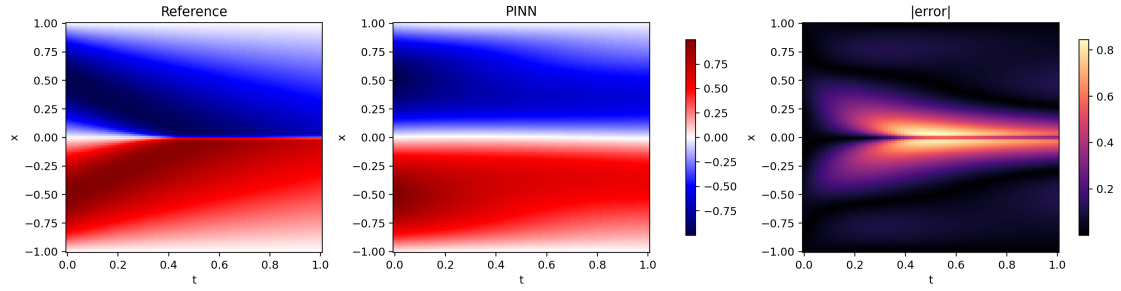


Figure 7.2: Burgers classical PINN prediction and absolute error.

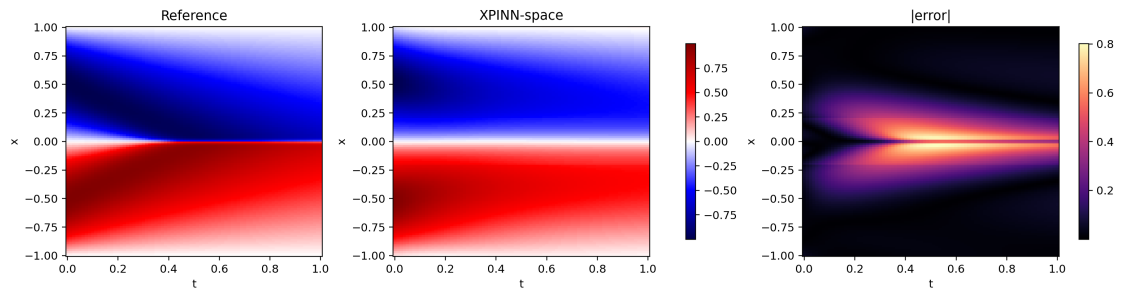


Figure 7.3: Burgers best model: space-split XPINN prediction and absolute error.

7.3 Burgers Comparison Plots

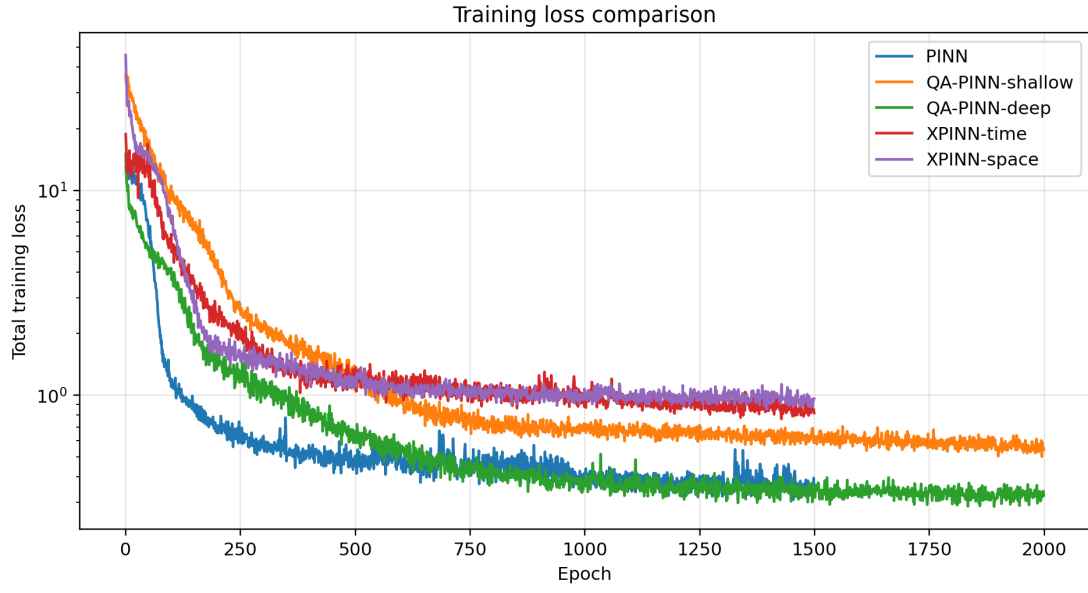


Figure 7.4: Burgers loss comparison across all five models.

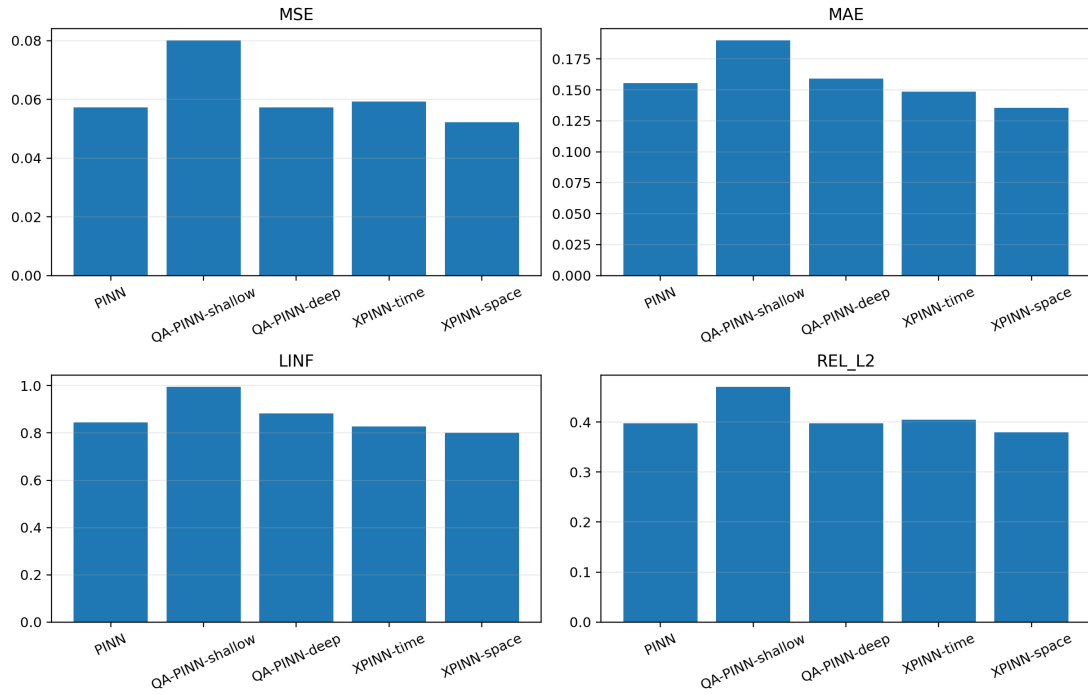


Figure 7.5: Burgers MSE, MAE, maximum error, and relative L^2 comparisons.

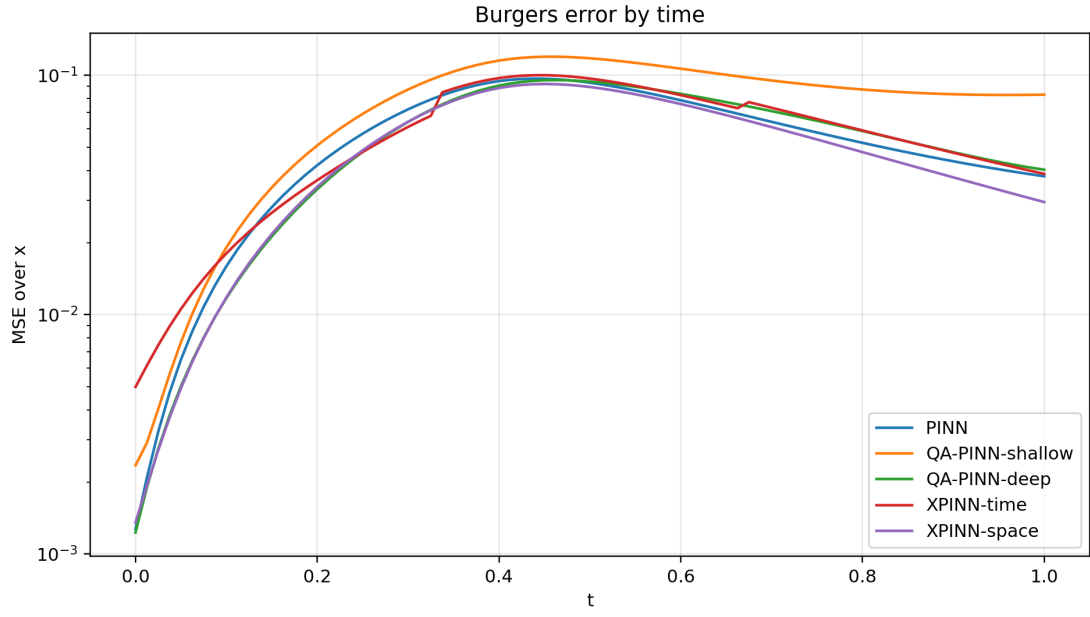


Figure 7.6: Burgers time-resolved MSE. This shows where in the time evolution each model accumulates error.

Model	Rel L2	MSE	Time (s)	Epochs	Params
PINN	0.3978	5.727e-02	6.9	1500	4,897
QA-PINN-shallow	0.4706	8.015e-02	43.8	2000	133
QA-PINN-deep	0.3976	5.720e-02	88.8	2000	7,315
XPINN-time	0.4046	5.924e-02	13.6	1500	10,323
XPINN-space	0.3798	5.219e-02	13.4	1500	10,323

Figure 7.7: Burgers summary table generated from the experiment output.

Chapter 8

Poisson Results

8.1 Numerical Table

Table 8.1: Poisson errors and cost metrics for the five active models.

Model	Rel. L^2	MSE	Time (s)	Epochs	Params
PINN	0.00566	7.75×10^{-6}	9.2	1500	4897
QA-PINN shallow	0.00346	2.91×10^{-6}	60.5	2000	133
QA-PINN deep	0.07411	1.33×10^{-3}	124.3	2000	7315
XPINN _x	0.01686	6.89×10^{-5}	19.0	1500	10323
XPINN _y	0.01664	6.71×10^{-5}	19.2	1500	10323

The best Poisson model is the shallow QA-PINN. This is an important result because it shows that the small quantum-assisted model can be highly parameter efficient for a smooth sinusoidal solution. The classical PINN is second best and is much faster than both QA-PINNs. The deep QA-PINN is less accurate for Poisson despite having more parameters, which suggests that deeper hybrid models can be harder to optimize. For Poisson, the exact residual identity

$$u_{xx}^* + u_{yy}^* + 2\pi^2 \sin(\pi x) \sin(\pi y) = 0 \quad (8.1)$$

makes the evaluation especially direct. The shallow QA-PINN minimizes

$$\mathcal{L}_P(\theta) = \frac{1}{N_f} \sum_i |r_\theta(x_i, y_i)|^2 + \lambda_b \frac{1}{N_b} \sum_i |u_\theta(x_i^b, y_i^b)|^2 \quad (8.2)$$

with only 133 trainable parameters, giving the lowest relative L^2 error in the table.

8.2 Poisson Prediction Plots

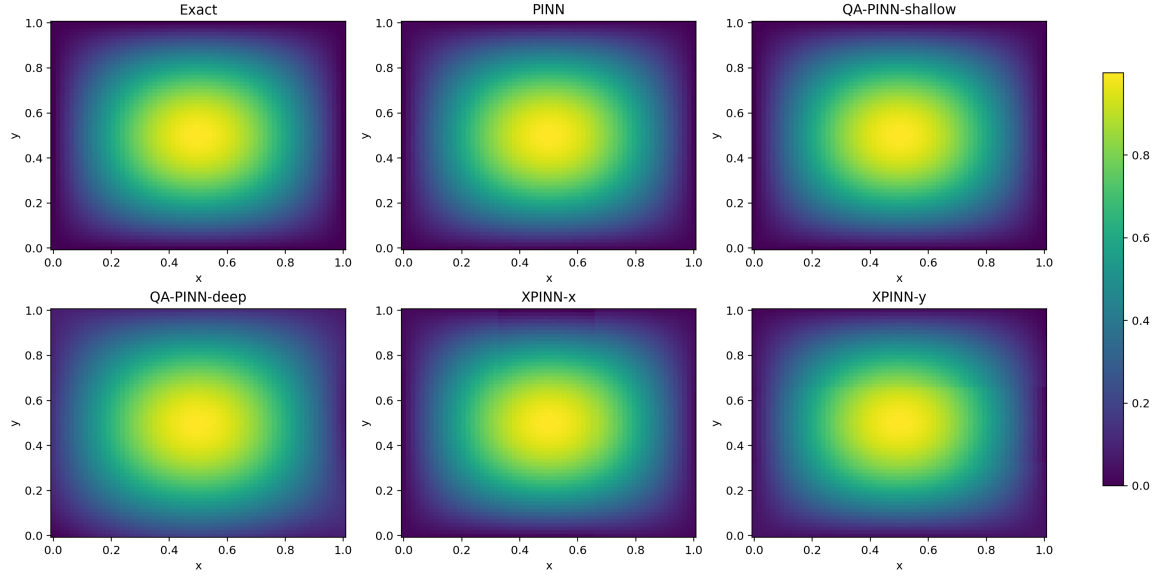


Figure 8.1: Poisson prediction montage for exact solution and all five models.

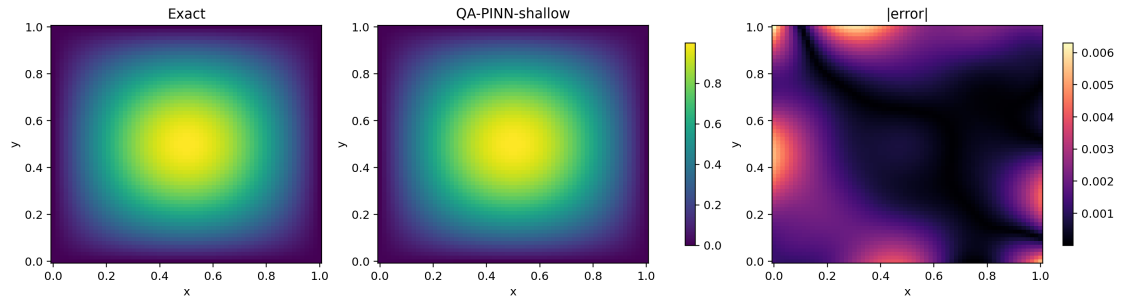


Figure 8.2: Poisson best model: shallow QA-PINN prediction and absolute error.

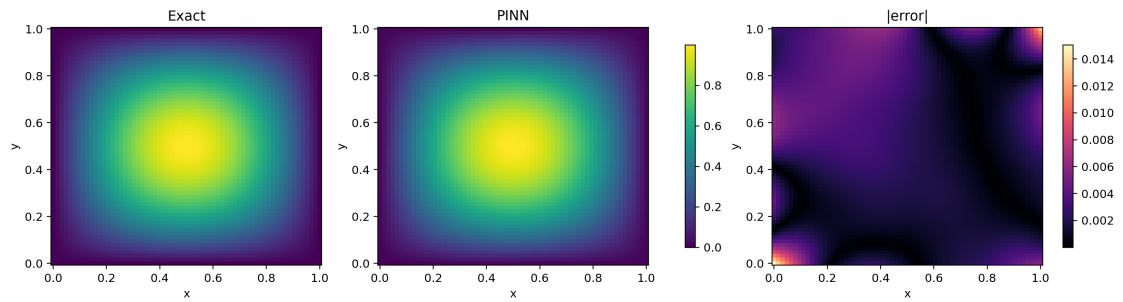


Figure 8.3: Poisson classical PINN prediction and absolute error.

8.3 Poisson Comparison Plots

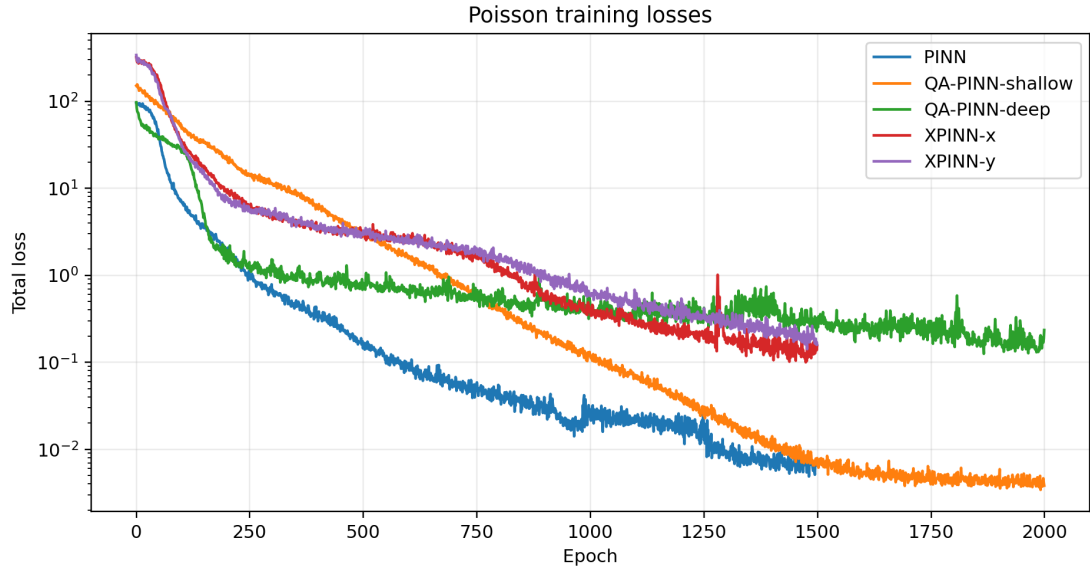


Figure 8.4: Poisson loss comparison across all five models.

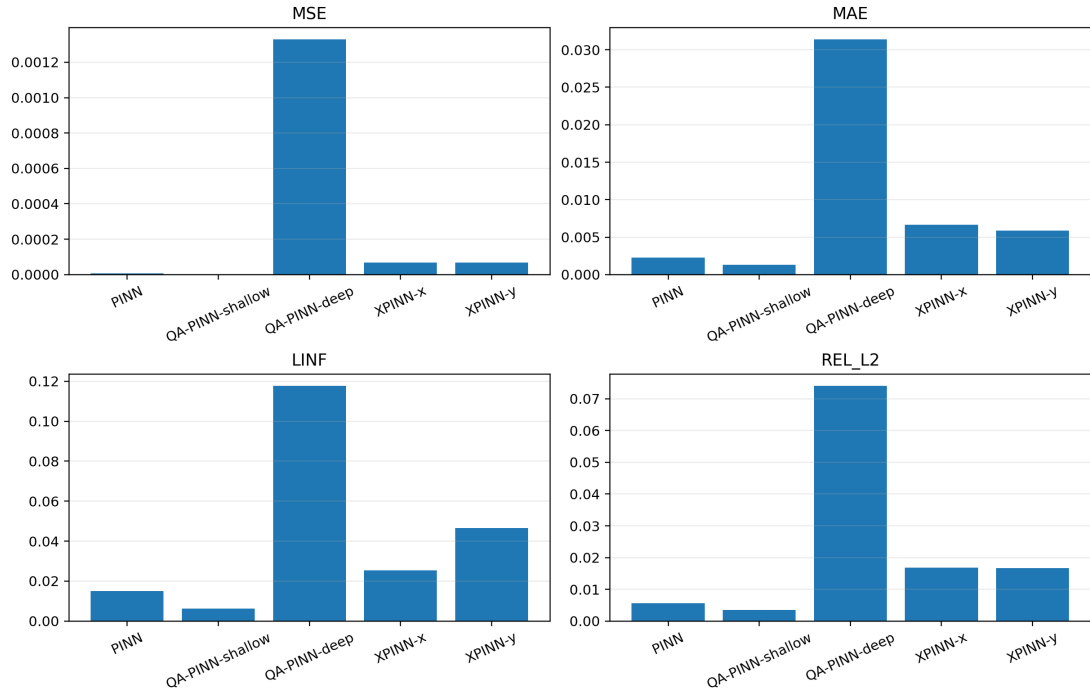


Figure 8.5: Poisson MSE, MAE, maximum error, and relative L^2 comparisons.

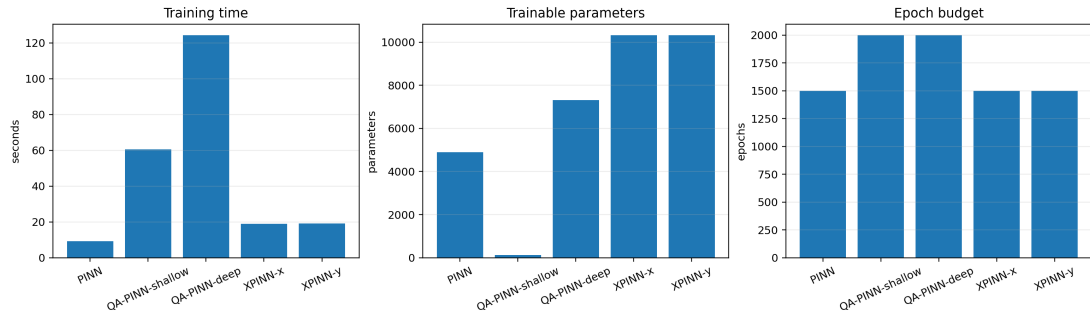


Figure 8.6: Poisson comparison of training time, trainable parameters, and epoch budgets.

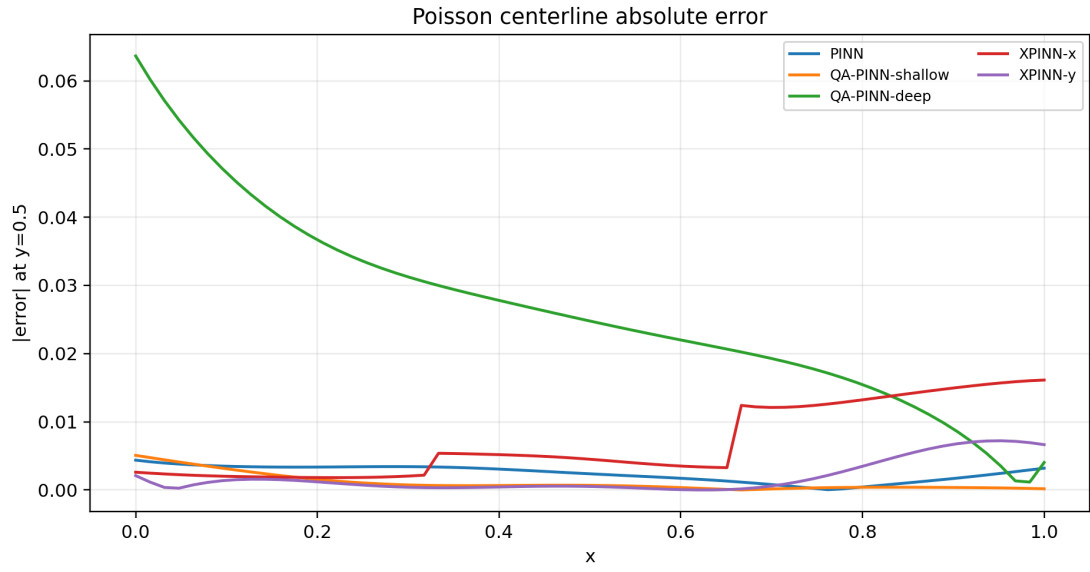


Figure 8.7: Poisson centerline absolute error at $y = 0.5$.

Chapter 9

Discussion

9.1 Burgers Interpretation

The Burgers results show that domain decomposition is useful when the split matches the physics. The space-split XPINN performs best because the central subdomain captures the region where the solution is hardest to approximate. The time-split XPINN is useful but slightly less effective. The deep QA-PINN nearly matches the classical PINN but requires much longer training time. The shallow QA-PINN is very small but lacks enough expressive capacity for the Burgers solution. The localized difficulty can be interpreted through the gradient energy

$$G(t) = \int_{-1}^1 |u_x(x, t)|^2 dx, \quad (9.1)$$

which becomes concentrated near the steepening region. A space split effectively assigns a separate approximation u_{θ_2} to this region, reducing the burden on a single global network.

9.2 Poisson Interpretation

The Poisson results show the opposite pattern. The shallow QA-PINN performs best, likely because the exact solution is sinusoidal and the quantum feature map naturally produces trigonometric behavior. The classical PINN is also very accurate and much faster. The XPINNs are accurate but not better than the global PINN because the Poisson solution is smooth and does not require domain specialization. The Poisson target is separable:

$$u^*(x, y) = X(x)Y(y), \quad X(x) = \sin(\pi x), \quad Y(y) = \sin(\pi y). \quad (9.2)$$

Since $\Delta u^* = X''Y + XY'' = -2\pi^2 XY$, the solution is globally smooth and does not create a localized interface advantage for XPINNs.

9.3 Accuracy Versus Efficiency

Efficiency is not only accuracy. A model with low error but very high training time may not be the best practical choice. For Burgers, the space-split XPINN gives the best accuracy

with moderate training time. For Poisson, the shallow QA-PINN gives the best accuracy with extremely few parameters but longer training time than the PINN. The classical PINN is therefore a very strong practical baseline for Poisson. One practical efficiency score is

$$S_{\text{time}} = \frac{\text{RelL2}}{T_{\text{train}}}, \quad (9.3)$$

while a parameter-efficiency view is

$$S_{\text{param}} = \text{RelL2} \times P. \quad (9.4)$$

These scores emphasize that the preferred architecture can change depending on whether the limiting resource is accuracy, time, or memory.

9.4 Role of Model Size

The parameter counts reveal a clear contrast. The shallow QA-PINN has only 133 trainable parameters. The deep QA-PINN has 7315. The classical PINN has 4897. The XPINNs have 10323 because each contains three subnetworks. Parameter count alone does not determine runtime: the shallow QA-PINN has very few parameters but is slower than the classical PINN because quantum circuit simulation is expensive. If a network has layer widths $d_0, d_1, \dots, d_L$, then

$$P = \sum_{\ell=1}^L (d_{\ell-1}d_{\ell} + d_{\ell}). \quad (9.5)$$

This formula explains the classical counts, while the QA-PINN count adds variational quantum angles:

$$P_{\text{total}} = P_{\text{classical}} + n_{\text{layers}}n_{\text{qubits}}. \quad (9.6)$$

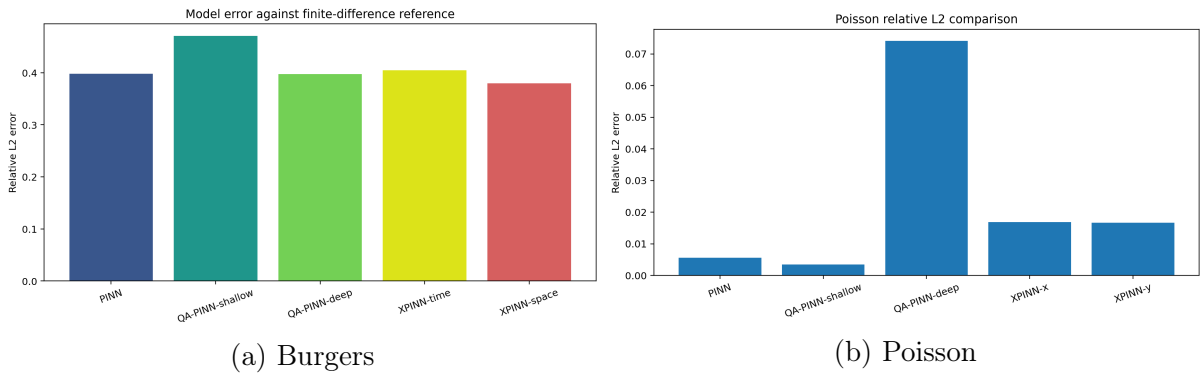


Figure 9.1: Relative L^2 summary across the two equations. The best architecture changes with the PDE.

Chapter 10

Limitations and Future Work

10.1 Limitations

The experiments were run on CPU. The QA-PINN simulations are especially expensive because each quantum layer is simulated classically. The reported results are therefore strong within the available compute budget but should not be treated as globally optimized hyperparameter searches. A full thesis-grade optimization study would test more widths, depths, learning rates, collocation densities, interface weights, qubit counts, and quantum ansatz choices.

Another limitation is that the Burgers reference is numerical rather than exact. The finite-difference reference is stable and consistent, but it is still an approximation. The Poisson reference is exact, which makes the Poisson error values more direct. The finite-difference timestep is constrained by convection and diffusion:

$$\Delta t \leq \min \left(C_{\text{cfl}} \frac{\Delta x}{\max |u|}, C_{\text{diff}} \frac{\Delta x^2}{\nu} \right). \quad (10.1)$$

Therefore the Burgers reference error contains both spatial and temporal discretization effects:

$$u_{\text{FD}} = u^* + \mathcal{O}(\Delta x^p) + \mathcal{O}(\Delta t^q). \quad (10.2)$$

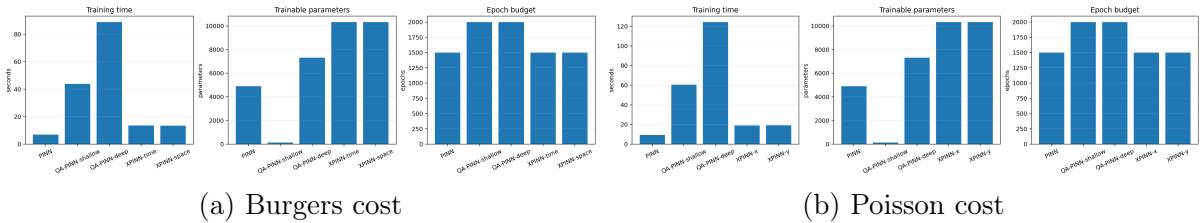


Figure 10.1: Training cost and model-size comparison. These plots motivate the limitations and future work.

10.2 Future Work

Future work should include GPU training, adaptive residual sampling, longer training schedules, and automated hyperparameter tuning. For Burgers, adaptive sampling near the steep-

ening region could improve all models. For XPINNs, different widths per subdomain could improve efficiency. For QA-PINNs, data re-uploading, more qubits, different entangling patterns, and quantum natural gradient methods could be explored. Adaptive sampling can be written as drawing future collocation points from a residual-weighted density

$$p(\mathbf{x}) = \frac{|r_\theta(\mathbf{x})|^\gamma}{\int_\Omega |r_\theta(\mathbf{s})|^\gamma d\mathbf{s}}, \quad \gamma > 0. \quad (10.3)$$

Quantum natural gradient methods would replace the Euclidean update by

$$\theta_{k+1} = \theta_k - \eta F(\theta_k)^{-1} \nabla_\theta \mathcal{L}(\theta_k), \quad (10.4)$$

where $F(\theta)$ is a quantum Fisher information or metric tensor approximation.

It would also be useful to compare the models against classical finite-difference and finite-element runtime more directly. PINNs are not always the fastest method for forward PDE solution; their value is strongest when data are sparse, geometry is irregular, inverse parameters are unknown, or mesh generation is difficult.

Chapter 11

Conclusion

This project compared five physics-informed neural models on the Burgers and Poisson equations: PINN, two XPINN splits, shallow QA-PINN, and deep QA-PINN. Tensor-network methods were removed from the final active comparison. The Burgers equation favored the space-split XPINN, which achieved the best relative L^2 error. The Poisson equation favored the shallow QA-PINN, which achieved the best relative L^2 error with only 133 trainable parameters.

The broader conclusion is that the best model depends on the equation. Burgers benefits from domain decomposition because its difficulty is spatially localized. Poisson benefits from a compact sinusoidal quantum-assisted representation, though the classical PINN remains an efficient and accurate baseline. Model evaluation must therefore include accuracy, training time, epochs, and parameter count. A model is not efficient simply because it is accurate, and it is not practical simply because it is small. The final comparison demonstrates that physics-informed learning is most meaningful when the architecture is chosen with the PDE structure in mind. The final decision rule can be summarized as

$$\text{best model} = \arg \min_m (\text{Rel}L2_m, T_m, P_m), \quad (11.1)$$

where the minimization is multi-objective rather than a single scalar ranking. Under pure relative- L^2 accuracy, the winners are

$$m_{\text{Burgers}}^* = \text{XPINN-space}, \quad m_{\text{Poisson}}^* = \text{QA-PINN-shallow}. \quad (11.2)$$

Model	Rel L2	MSE	Time (s)	Epochs	Params
PINN	0.3978	5.727e-02	6.9	1500	4.897
QA-PINN-shallow	0.4706	8.015e-02	43.8	2000	133
QA-PINN-deep	0.3976	5.720e-02	88.8	2000	7.315
XPINN-time	0.4046	5.924e-02	13.6	1500	10.323
XPINN-space	0.3798	5.219e-02	13.4	1500	10.323

(a) Burgers summary

Model	Rel L2	MSE	Time (s)	Epochs	Params
PINN	0.0057	7.754e-06	9.2	1500	4.897
QA-PINN-shallow	0.0035	2.908e-06	60.5	2000	133
QA-PINN-deep	0.0741	1.331e-03	124.3	2000	7.315
XPINN-x	0.0369	6.889e-05	19.0	1500	10.323
XPINN-y	0.0166	6.719e-05	19.2	1500	10.323

(b) Poisson summary

Figure 11.1: Final summary tables generated directly from the verified experiment outputs.

Bibliography

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, 378, 686–707, 2019. doi: [10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045).
- [2] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, “Physics-informed machine learning,” *Nature Reviews Physics*, 3, 422–440, 2021. doi: [10.1038/s42254-021-00314-5](https://doi.org/10.1038/s42254-021-00314-5).
- [3] A. D. Jagtap, E. Kharazmi, and G. E. Karniadakis, “Conservative physics-informed neural networks on discrete domains for conservation laws,” *Computer Methods in Applied Mechanics and Engineering*, 365, 113028, 2020. doi: [10.1016/j.cma.2020.113028](https://doi.org/10.1016/j.cma.2020.113028).
- [4] A. D. Jagtap and G. E. Karniadakis, “Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations,” *Communications in Computational Physics*, 28(5), 2002–2041, 2020. doi: [10.4208/cicp.OA-2020-0164](https://doi.org/10.4208/cicp.OA-2020-0164).
- [5] C. Trahan, M. Loveland, and S. Dent, “Quantum Physics-Informed Neural Networks,” *Entropy*, 26(8), 649, 2024. doi: [10.3390/e26080649](https://doi.org/10.3390/e26080649).
- [6] A. Farea, S. Khan, and M. S. Celebi, “QCPINN: quantum-classical physics-informed neural networks for solving PDEs,” *Machine Learning: Science and Technology*, 6(4), 2025. doi: [10.1088/2632-2153/ae1c91](https://doi.org/10.1088/2632-2153/ae1c91).
- [7] F. Y. Leong, W. B. Ewe, S. B. Q. Tran, Z. Zhang, and J. Y. Khoo, “Hybrid quantum physics-informed neural network: Towards efficient learning of high-speed flows,” *Computers and Fluids*, 301, 106782, 2025. doi: [10.1016/j.compfluid.2025.106782](https://doi.org/10.1016/j.compfluid.2025.106782).
- [8] PennyLane documentation, “qml.qnn.TorchLayer,” docs.pennylane.ai.
- [9] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, “DeepXDE: A deep learning library for solving differential equations,” *SIAM Review*, 63(1), 208–228, 2021.
- [10] J. Shah, R. Lineswala, and A. Chopra, “Benchmarking Quantum-Assisted PINN (QA-PINN) for Computational Fluid Dynamics,” local project PDF, BosonQ Psi.